

the other (the private key) remains secret. The surprising strength of this system lies in the use of one-way mathematical functions, such as factoring large numbers or solving discrete logarithms, which are easy to compute in one direction but practically impossible to reverse without the private key. Despite appearing to ‘leak’ half of the key, the security holds because knowing the public key does not enable an attacker to efficiently derive the private key or decrypt messages.

## A first glance at public-key cryptography

Diffie-Hellman key exchange (DH), introduced in 1976, was groundbreaking as the first practical method for securely exchanging cryptographic keys over an insecure channel. While DH itself is not an encryption scheme, it established the fundamental idea of using modular arithmetic and hard mathematical problems (like discrete logarithms) for secure communication. The success of DH in demonstrating the feasibility of public-key cryptography directly paved the way for RSA and its widespread adoption. RSA was developed by Ron Rivest, Adi Shamir, and Leonard Adleman in 1977 at MIT, and the algorithm is named after their initials. RSA built upon similar mathematical principles, particularly modular exponentiation, but instead of key exchange, it provided a complete encryption system using asymmetric cryptography.

Since RSA can be somewhat counter-intuitive and difficult to grasp at first, this article will focus on a simplified, toy model known as Kid-RSA (KRSA), which captures the core ideas of RSA in an easier-to-understand form. KRSA is a simplified, insecure public-key cipher published in 1997, designed for educational purposes. Cybersecurity researchers often feel that learning KRSA gives insight into RSA and other public-key ciphers, analogous to simplified data encryption standard (S-DES).

Now, consider the SageMath program named ‘krsa30.sage’ shown below, which implements the KRSA encryption scheme. The program demonstrates the encryption and decryption of a single number, 15. However, it is important to note that encoding schemes like ASCII or Unicode represent characters as numbers. Therefore, with slight modifications, this program can be extended to encrypt entire messages by processing their numerical representations.

```
a = 3
n = 17
b = inverse_mod(a, n)
M = 5
C = (M * a) % n
print("Encrypted ciphertext:", C)
decrypted = (C * b) % n
print("Decrypted plaintext:", decrypted)
```

Now, let us understand how the program works. The program uses modular multiplication over a prime modulus

to implement KRSA. The encryption key  $a$  is set to 3, and the modulus  $n$  is chosen as 17 (a prime number to ensure the existence of modular inverses). The decryption key  $b$  is computed as the modular inverse of  $a$  modulo  $n$ . A plaintext value  $M = 5$  is encrypted as  $C = (M * a) \% n$ , and the ciphertext is then decrypted using  $decrypted = (C * b) \% n$ . The decryption process successfully recovers the original plaintext value, demonstrating the reversibility of the encryption when working in a finite field defined by a prime modulus.

In this example, the public key consists of the pair  $(a, n)$ , where  $a=3$  is the encryption multiplier and  $n=17$  is the prime modulus. The private key is derived by computing the modular inverse of  $a$  modulo  $n$ , resulting in  $b=inverse\_mod(a, n)$ , which equals 6. Therefore, the private key is the pair  $(b, n)$ , which in this case is  $(6, 17)$ .

Now, let us consider a scenario where Alice wants to send a message to Bob. (Recall that Alice and Bob are classical characters commonly used in cybersecurity textbooks and examples.) To send her message securely, Alice uses Bob’s public key, which in this case is  $(3, 17)$ , to encrypt the plaintext. She then sends the resulting ciphertext to Bob. Since only Bob knows the private key  $(6, 17)$ , he alone can decrypt the ciphertext and retrieve the original message.

It is important to note that while anyone can encrypt messages using Bob’s public key, only Bob can decrypt them using his private key. This eliminates the need to exchange a secret key over a potentially insecure channel — a major challenge in symmetric-key encryption. In fact, Bob can even publish his public key openly, for example on social media platforms like Facebook, Instagram, or Twitter, without compromising the security of the communication.

Upon executing the program ‘krsa30.sage’, the output shown in Figure 3 is obtained. As shown, the original plaintext value of 5 is successfully recovered during decryption.

```
# sage krsa30.sage
Encrypted ciphertext: 15
Decrypted plaintext: 5
```

Figure 3: Kid-RSA implementation

Now, it is time to wrap up our discussion. In this article, we explored how cybersecurity could be significantly impacted in the near future by advancements in quantum computing. We also introduced the concept of cryptanalysis and concluded with the basics of public-key cryptography through a simplified model called Kid-RSA (KRSA). In the next article in this series, we will continue our exploration of public-key cryptography, with a special focus on understanding and implementing RSA. 

 By: Dr Deepu Benson

The author is a free software enthusiast, and his area of interest is theoretical computer science.